

Ordem Prática de Desenvolvimento do MVP

Sistema Web de Divisão de Despesas com Django

Objetivo: desenvolver uma primeira versão funcional, segura e publicável, validando primeiro as regras matemáticas, depois o banco de dados e, por fim, as telas.

Escopo do MVP: cadastro e login, grupos, participantes, despesas com um único pagador, divisão igualitária, cálculo de saldos, sugestão de transferências e histórico.

Fora do MVP: múltiplos pagadores, porcentagens, cotas, anexos, Pix, aplicativo móvel, integração bancária e funcionamento offline.

Documento de planejamento técnico

Índice

Visão geral das etapas	4
1. Preparar o ambiente de desenvolvimento	5
Criar a pasta e o ambiente virtual	5
Instalar o Django e registrar dependências	5
Iniciar o Git	5
2. Validar as regras de negócio em Python puro	6
2.1 Divisão igualitária	6
2.2 Testar diferenças de centavos	6
2.3 Calcular saldos	6
2.4 Gerar transferências	7
3. Criar o projeto Django	8
4. Criar as aplicações Django	9
5. Modelar o banco de dados	10
5.1 Usuário	10
5.2 Grupo	10
5.3 Participante	10
5.4 Despesa	10
5.5 Participação na despesa	11
6. Configurar o Django Admin	12
7. Implementar autenticação	13
8. Implementar o CRUD de grupos	14
9. Implementar participantes	15
10. Implementar o cadastro de despesas	16
Fluxo ao salvar	16
11. Criar uma camada de serviços	17

12. Implementar o histórico de despesas	18
13. Implementar o cálculo de saldos do grupo	19
14. Implementar as transferências sugeridas	20
15. Construir os templates e aplicar Bootstrap	21
Telas mínimas	21
16. Implementar permissões	22
17. Implementar validações do domínio	23
18. Criar testes automatizados	24
Testes das regras de negócio	24
Testes dos models	24
Testes das views	24
Teste completo do fluxo	24
19. Revisar o MVP antes de publicar	25
20. Preparar o banco de produção	26
21. Preparar arquivos estáticos e servidor	27
22. Publicar o MVP	28
Ordem resumida por entregas	29
Entrega 1 - Motor de cálculo	29
Entrega 2 - Estrutura do Django	29
Entrega 3 - Grupos	29
Entrega 4 - Despesas	29
Entrega 5 - Acerto de contas	29
Entrega 6 - MVP publicado	29
Recomendação de foco	30

Visão geral das etapas

1. Preparar o ambiente
2. Validar as regras de negócio em Python
3. Criar o projeto Django
4. Modelar o banco de dados
5. Configurar o Django Admin
6. Implementar usuários e autenticação
7. Implementar grupos e participantes
8. Implementar despesas
9. Calcular saldos
10. Gerar transferências sugeridas
11. Construir as telas
12. Implementar permissões e validações
13. Criar testes
14. Preparar publicação
15. Publicar o MVP

A sequência reduz riscos: primeiro se valida o cálculo central; depois se representa o domínio no banco; só então se desenvolvem formulários e telas.

1. Preparar o ambiente de desenvolvimento

Antes de escrever funcionalidades, prepare uma base organizada e reproduzível.

Criar a pasta e o ambiente virtual

```
mkdir divisao_despesas
cd divisao_despesas
py -3.11 -m venv .venv
.venv\Scripts\Activate.ps1
```

Instalar o Django e registrar dependências

```
python -m pip install --upgrade pip
pip install django
pip freeze > requirements.txt
```

Iniciar o Git

```
git init
```

Crie um arquivo .gitignore:

```
.venv/
__pycache__/
*.py[codz]
.env
db.sqlite3
.vscode/
```

Resultado esperado: ambiente virtual isolado, dependências registradas e repositório preparado para versionamento.

2. Validar as regras de negócio em Python puro

Antes de trabalhar com Django, formulários e banco, desenvolva as funções matemáticas isoladamente. A principal complexidade do sistema está nas regras de divisão e liquidação, não no CRUD.

```
divisao_despesas/  
└─ regras/  
    ├── __init__.py  
    ├── divisao.py  
    └─ test_divisao.py
```

2.1 Divisão igualitária

A função recebe o valor total e a lista de participantes e devolve quanto cada pessoa deve pagar, garantindo o fechamento em centavos.

```
from decimal import Decimal, ROUND_DOWN  
  
def dividir_igualmente(valor_total: Decimal, participantes: list[str]) -> dict[str, Decimal]:  
    if valor_total <= Decimal("0.00"):  
        raise ValueError("0 valor deve ser maior que zero.")  
    if not participantes:  
        raise ValueError("A despesa deve ter participantes.")  
  
    quantidade = len(participantes)  
    valor_base = (valor_total / quantidade).quantize(  
        Decimal("0.01"), rounding=ROUND_DOWN  
    )  
    valores = {p: valor_base for p in participantes}  
    restante = valor_total - (valor_base * quantidade)  
    centavos_restantes = int(restante / Decimal("0.01"))  
  
    for participante in participantes[:centavos_restantes]:  
        valores[participante] += Decimal("0.01")  
  
    return valores
```

2.2 Testar diferenças de centavos

```
resultado = dividir_igualmente(  
    Decimal("100.00"),  
    ["Ana", "Bruno", "Carlos"],  
)
```

Participante	Valor devido
Ana	R\$ 33,34
Bruno	R\$ 33,33
Carlos	R\$ 33,33

2.3 Calcular saldos

```
saldo = total_pago - total_devido
```

Saldo positivo significa valor a receber; saldo negativo significa valor a pagar; saldo zero significa que a pessoa está quitada.

2.4 Gerar transferências

Separe credores e devedores e gere instruções até que todos os saldos sejam zerados.

Bruno paga R\$ 100,00 para Ana

Carlos paga R\$ 50,00 para Ana

Critério de conclusão: dividir valores com centavos, calcular saldos, gerar transferências e verificar que a soma final dos saldos é zero.

3. Criar o projeto Django

Depois de validar a lógica principal, crie o projeto Django na pasta atual.

```
django-admin startproject config .
```

O ponto final significa: criar o projeto na pasta em que o comando está sendo executado.

```
divisao_despesas/  
├─ manage.py  
├─ config/  
│   ├── __init__.py  
│   ├── settings.py  
│   ├── urls.py  
│   ├── asgi.py  
│   └─ wsgi.py  
├─ regras/  
├─ requirements.txt  
└─ .gitignore
```

```
python manage.py runserver
```

Acesse <http://127.0.0.1:8000/> e confirme a página inicial do Django.

```
git add .  
git commit -m "Cria estrutura inicial do projeto Django"
```


4. Criar as aplicações Django

Separe o projeto em aplicações relacionadas aos principais domínios.

```
python manage.py startapp usuarios
python manage.py startapp grupos
python manage.py startapp despesas
```

```
divisao_despesas/
├─ config/
├─ usuarios/
├─ grupos/
├─ despesas/
├─ regras/
└─ manage.py
```

Registre as aplicações em `INSTALLED_APPS` no arquivo `config/settings.py`.

```
INSTALLED_APPS = [
    "django.contrib.admin",
    "django.contrib.auth",
    "django.contrib.contenttypes",
    "django.contrib.sessions",
    "django.contrib.messages",
    "django.contrib.staticfiles",
    "usuarios",
    "grupos",
    "despesas",
]
```

5. Modelar o banco de dados

Crie os modelos em uma ordem compatível com as dependências: usuário, grupo, participante, despesa e participação na despesa.

5.1 Usuário

Para o MVP, use inicialmente o usuário padrão do Django, que já oferece autenticação, senha segura, permissões e integração com o Admin.

```
from django.contrib.auth.models import User
```

5.2 Grupo

```
class Grupo(models.Model):
    nome = models.CharField(max_length=100)
    descricao = models.TextField(blank=True)
    criado_por = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="grupos_criados",
    )
    criado_em = models.DateTimeField(auto_now_add=True)
    ativo = models.BooleanField(default=True)
```

5.3 Participante

Permita participantes sem conta, vinculando opcionalmente um usuário cadastrado.

```
class Participante(models.Model):
    grupo = models.ForeignKey(Grupo, on_delete=models.CASCADE, related_name="participantes")
    usuario = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.SET_NULL,
        null=True,
        blank=True,
        related_name="participacoes",
    )
    nome = models.CharField(max_length=100)
    email = models.EmailField(blank=True)
    ativo = models.BooleanField(default=True)
```

5.4 Despesa

```
class Despesa(models.Model):
    grupo = models.ForeignKey(Grupo, on_delete=models.CASCADE, related_name="despesas")
    descricao = models.CharField(max_length=200)
    valor_total = models.DecimalField(max_digits=12, decimal_places=2)
    data_despesa = models.DateField()
    categoria = models.CharField(max_length=100, blank=True)
    observacao = models.TextField(blank=True)
    pagador = models.ForeignKey(
        Participante,
        on_delete=models.PROTECT,
        related_name="despesas_pagas",
    )
    criado_por = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.PROTECT,
        related_name="despesas_criadas",
    )
    criado_em = models.DateTimeField(auto_now_add=True)
```

5.5 Participação na despesa

```
class ParticipacaoDespesa(models.Model):
    despesa = models.ForeignKey(
        Despesa,
        on_delete=models.CASCADE,
        related_name="participacoes",
    )
    participante = models.ForeignKey(
        Participante,
        on_delete=models.PROTECT,
        related_name="despesas_participadas",
    )
    valor_devido = models.DecimalField(max_digits=12, decimal_places=2)
```

Essa tabela registra o resultado concreto da divisão e permite conservar o histórico.

```
python manage.py makemigrations
python manage.py migrate
```

6. Configurar o Django Admin

Antes de construir telas públicas, use o Admin para validar os modelos e relacionamentos.

```
python manage.py createsuperuser

from django.contrib import admin
from .models import Grupo, Participante

admin.site.register(Grupo)
admin.site.register(Participante)
```

Registre também Despesa e ParticipacaoDespesa no aplicativo de despesas.

Teste manual: cadastre um usuário, um grupo, três participantes, uma despesa e os respectivos valores devidos.

7. Implementar autenticação

O sistema deve impedir o acesso de usuários não autenticados e oferecer cadastro, login e logout.

```
urlpatterns = [  
    path("admin/", admin.site.urls),  
    path("contas/", include("django.contrib.auth.urls")),  
]  
  
templates/  
└─ registration/  
    └─ login.html
```

Resultado esperado: o usuário consegue criar conta, entrar, sair e acessar somente áreas autorizadas.

8. Implementar o CRUD de grupos

Comece pelo fluxo estrutural mais simples do sistema.

- Criar um grupo.
- Listar os grupos do usuário.
- Visualizar os detalhes.
- Editar.
- Encerrar ou excluir.

```
/grupos/  
/grupos/novo/  
/grupos/1/  
/grupos/1/editar/  
/grupos/1/excluir/
```

Ordem interna recomendada: Listar → Criar → Detalhar → Editar → Excluir

Na tela de detalhes, mostre inicialmente o nome, a descrição, os participantes, as despesas e o total gasto.

9. Implementar participantes

- Adicionar participante.
- Listar participantes.
- Editar nome e email.
- Desativar participante.
- Impedir duplicidade.

`/grupos/1/participantes/novo/`

`/grupos/1/participantes/4/editar/`

`/grupos/1/participantes/4/remove/`

Não exclua diretamente um participante associado a despesas. Para preservar o histórico, prefira marcá-lo como inativo.

10. Implementar o cadastro de despesas

Essa é a etapa central do sistema.

Campo	Finalidade
Descrição	Identificar o gasto
Valor	Registrar o total
Data	Situar a despesa
Categoria	Facilitar organização
Pagador	Indicar quem desembolsou
Participantes	Indicar quem suporta o custo
Observação	Registrar detalhes opcionais

A forma de divisão do MVP será sempre igualitária.

Fluxo ao salvar

- Validar os dados.
- Confirmar que o pagador pertence ao grupo.
- Confirmar que os participantes pertencem ao grupo.
- Criar a despesa.
- Executar a divisão igualitária.
- Criar uma ParticipacaoDespesa para cada participante.
- Verificar que a soma dos valores devidos é igual ao total.

Use uma transação atômica para impedir registros incompletos.

```
from django.db import transaction

@transaction.atomic
def criar_despesa_com_divisao(...):
    ...
```


11. Criar uma camada de serviços

Evite colocar toda a regra de negócio dentro das views.

```
despesas/  
└─ services.py  
  
@transaction.atomic  
def registrar_despesa(*, grupo, descricao, valor_total, data_despesa,  
                     pagador, participantes, criado_por):  
    valores = dividir_igualmente(valor_total, list(participantes))  
    despesa = Despesa.objects.create(  
        grupo=grupo,  
        descricao=descricao,  
        valor_total=valor_total,  
        data_despesa=data_despesa,  
        pagador=pagador,  
        criado_por=criado_por,  
    )  
    ParticipacaoDespesa.objects.bulk_create([  
        ParticipacaoDespesa(  
            despesa=despesa,  
            participante=p,  
            valor_devido=valores[p],  
        )  
        for p in participantes  
    ])  
    return despesa
```

View recebe a requisição → Form valida → Service executa a regra → ORM persiste → Template apresenta.

12. Implementar o histórico de despesas

Data	Descrição	Valor	Pagador
20/07/2026	Restaurante	R\$ 240,00	Thiago
21/07/2026	Combustível	R\$ 180,00	Ana

- Visualizar detalhes.
- Editar.
- Excluir.

Ao alterar valor, participantes ou regra de divisão, recalcule os valores devidos. No MVP, pode-se apagar as participações antigas e recriá-las dentro de uma transação.

13. Implementar o cálculo de saldos do grupo

total pago = soma das despesas pagas pela pessoa

total devido = soma das participações da pessoa

saldo = total pago - total devido

Participante	Pago	Devido	Saldo
Ana	R\$ 300,00	R\$ 150,00	R\$ 150,00
Bruno	R\$ 50,00	R\$ 150,00	-R\$ 100,00
Carlos	R\$ 100,00	R\$ 150,00	-R\$ 50,00

Coloque o cálculo em `despesas/services.py` ou em um módulo específico como `despesas/services/saldos.py`.

Ana deve receber R\$ 150,00

Bruno deve pagar R\$ 100,00

Carlos deve pagar R\$ 50,00

14. Implementar as transferências sugeridas

Use os saldos para gerar transferências entre devedores e credores.

```
[  
  {"devedor": "Bruno", "credor": "Ana", "valor": Decimal("100.00")},  
  {"devedor": "Carlos", "credor": "Ana", "valor": Decimal("50.00")},  
]
```

Bruno paga R\$ 100,00 para Ana
Carlos paga R\$ 50,00 para Ana

No MVP, as sugestões podem ser calculadas toda vez que a tela for aberta; não precisam ser armazenadas no banco.

15. Construir os templates e aplicar Bootstrap

Somente depois do fluxo principal funcionar, dedique mais tempo à apresentação.

```
templates/  
├─ base.html  
├─ registration/  
├─ grupos/  
└─ despesas/
```

- Barra de navegação.
- Formulários.
- Tabelas.
- Mensagens de sucesso e erro.
- Cartões.
- Botões.
- Layout responsivo.

Telas mínimas

1. Login
2. Painel com grupos
3. Detalhes do grupo
4. Nova despesa
5. Detalhes da despesa
6. Acerto de contas

16. Implementar permissões

Não basta esconder botões no HTML. A validação deve acontecer também nas views e nos serviços.

- Exigir usuário autenticado.
- Verificar autorização para visualizar o grupo.
- Confirmar que participantes pertencem ao grupo.
- Confirmar que o pagador pertence ao grupo.
- Confirmar que a despesa pertence ao grupo acessado.
- Restringir edição e exclusão.

```
grupo = get_object_or_404(
    Grupo,
    id=grupo_id,
    participantes__usuario=request.user,
)
```

17. Implementar validações do domínio

- Valor total maior que zero.
- Pelo menos um participante.
- Pagador pertencente ao grupo.
- Participantes pertencentes ao grupo.
- Soma dos valores devidos igual ao valor total.
- Tratamento correto dos centavos.

Também impeça despesas em grupos encerrados, edição de dados de terceiros, participantes duplicados e exclusões sem confirmação.

18. Criar testes automatizados

Testes das regras de negócio

- Divisão exata.
- Divisão com centavos.
- Lista vazia.
- Valor zero.
- Saldo positivo e negativo.
- Múltiplos credores e devedores.
- Soma final igual a zero.

Testes dos models

- Criação de grupo e participante.
- Relacionamentos.
- Restrição de participante único na despesa.

Testes das views

- Usuário não autenticado.
- Usuário sem acesso.
- Criação válida.
- Formulário inválido.
- Edição e exclusão.

Teste completo do fluxo

Criar usuário → criar grupo → adicionar participantes → criar despesa → calcular saldos → gerar transferências

19. Revisar o MVP antes de publicar

- Cadastro, login e logout funcionando.
- Criação de grupos e participantes.
- Cadastro, edição e exclusão de despesas.
- Divisão de centavos fechando corretamente.
- Saldos recalculados após alterações.
- Transferências geradas.
- Bloqueio de acesso a grupos de terceiros.
- Telas utilizáveis em celular.

20. Preparar o banco de produção

Use SQLite durante o desenvolvimento e PostgreSQL na publicação.

```
pip install psycopg[binary]
pip freeze > requirements.txt
```

Coloque em variáveis de ambiente: SECRET_KEY, dados do banco, modo de depuração e hosts permitidos. Nunca envie senhas para o Git.

21. Preparar arquivos estáticos e servidor

- Configurar DEBUG = False.
- Definir ALLOWED_HOSTS.
- Definir STATIC_ROOT.
- Coletar arquivos estáticos.
- Configurar servidor WSGI.

```
python manage.py collectstatic  
pip install gunicorn  
gunicorn config.wsgi
```

No Windows, durante o desenvolvimento, continue usando `python manage.py runserver`.

22. Publicar o MVP

- repositório GitHub
- serviço de hospedagem
- variáveis de ambiente
- banco PostgreSQL
- migrations
- arquivos estáticos
- aplicação disponível na web

```
python manage.py migrate
python manage.py createsuperuser
```

Depois da publicação, teste novamente o fluxo principal em ambiente real.

Ordem resumida por entregas

Entrega 1 - Motor de cálculo

- Divisão igualitária
- Tratamento de centavos
- Cálculo de saldos
- Transferências sugeridas
- Testes

Entrega 2 - Estrutura do Django

- Projeto Django
- Aplicações
- Models
- Migrations
- Admin

Entrega 3 - Grupos

- Login
- Cadastro
- CRUD de grupos
- Participantes
- Permissões

Entrega 4 - Despesas

- Cadastro
- Divisão igualitária
- Histórico
- Edição
- Exclusão

Entrega 5 - Acerto de contas

- Totais pagos
- Totais devidos
- Saldos
- Transferências

Entrega 6 - MVP publicado

- Bootstrap
- Responsividade
- Testes finais
- PostgreSQL
- Segurança
- Hospedagem

Recomendação de foco

Durante o desenvolvimento, evite adicionar funcionalidades fora do MVP. Só avance quando o fluxo anterior estiver funcionando e testado.

regra funcionando → teste automatizado → persistência no banco → formulário → tela → permissão

Essa sequência evita construir várias telas sobre uma regra de negócio ainda instável e reduz retrabalho.